Практическое занятие №5. Создание CRUD-приложения. Модели ответов и обработка ошибок

Модели ответов служат шаблонами для возврата данных из пути маршрута API. Они построены на **Pydantic** для правильной обработки ответов на запросы, отправленные на сервер. Обработка ошибок включает методы и действия, связанные с обработкойошибок в приложении. Эти методы включают возврат адекватных кодов состояния ошибки и сообщений об ошибках.

Цель – создание CRUD-приложения в FastAPI и модели ответов и их обработки.

Задачи работы:

- узнать, что такое ответ и из чего он состоит;
- узнать что такое обработка ошибок и о том, как обрабатывать ошибки в вашемприложении FastAPI;
- узнаеть, как создавать модели ответов на запросыс помощью Pydantic.

Разделы:

- Создание простого CRUD приложения
- Ответы в FastAPI
- Построение модели ответа
- Обработка ошибок

1.6. Создание простого CRUD-приложения

Мы создали маршруты для **создания** и **получения** задач. Построим маршруты для **обновления** и **удаления** добавленных задач. Начнем с создания модели тела запроса для маршрута UPDATE в `model.py`:

```
class TodoItem(BaseModel):
    item: str

    class Config:
        schema_extra = {
            "example": {
                "item": "Read the next chapter of the book"
            }
        }
```

Далее давайте напишем маршрут для обновления задачи в `todo.py`:

```
from fastapi import APIRouter, Path
from model import Todo, TodoItem

todo_router = APIRouter()
todo_list = []

@todo_router.post("/todo")
async def add_todo(todo: Todo) -> dict:
    todo_list.append(todo)
    return {
        "message": "Todo added successfully."
    }
```

```
@todo_router.get("/todo")
async def retrieve_todo() -> dict:
   return {
       "todos": todo_list
   }


@todo_router.get("/todo/{todo_id}")
async def get_single_todo(todo_id: int = Path(..., title="The
ID of the todo to retrieve")) -> dict:
   for todo in todo_list:
       if todo.id == todo_id:
           return {
               "todo": todo
           }
   return {
       "message": "Todo with supplied ID doesn't exist."
   }

@todo_router.put("/todo/{todo_id}")
async def update_todo(todo_data: TodoItem, todo_id: int =
Path(..., title="The ID of the todo to be updated")) -> dict:
   for todo in todo_list:
       if todo.id == todo_id:
           todo.item = todo_data.item
           return {
               "message": "Todo updated successfully."
           }
   return {
       "message": "Todo with supplied ID doesn't exist."
    }
```

Протестируем новый маршрут. Во-первых, давайте добавим задачу:

```
(venv)$ curl -X POST \
  'http://127.0.0.1:8000/todo' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 1,
  "item": "Example Schema!"
}'
```

Вот ответ:

```
(venv)$ {
  "message": "Todo added successfully."
}
```

Далее давайте обновим задачу, отправив запрос `PUT`:

```
(venv)$ curl -X PUT \
  'http://127.0.0.1:8000/todo/1' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "item": "Read the next chapter of the book."
}'
```

Вот ответ:

```
(venv)$ {
  "message": "Todo updated successfully."
}
```

Давайте проверим, что наша задача действительно была обновлена:

```
(venv)$ curl -X GET \
  'http://127.0.0.1:8000/todo/1' \
  -H 'accept: application/json'
```

Вот ответ:

```
(venv)$ {
  "todo": {
    "id": 1,
    "item": "Read the next chapter of the book"
  }
}
```

Из возвращенного ответа мы видим, что задача успешно обновлена. Теперь давайтесоздадим маршрут для удаления задачи и всех задач.

В `todo.py`, обновите маршруты:

```
@todo_router.delete("/todo/{todo_id}")
async def delete_single_todo(todo_id: int) -> dict:
   for index in range(len(todo_list)):
       todo = todo_list[index]
       if todo.id == todo_id:
           todo_list.pop(index)return {"message": "Todo deleted
           successfully."}
   return {"message": "Todo with supplied ID doesn't exist."}


@todo_router.delete("/todo")
async def delete_all_todo() -> dict:
   todo_list.clear()
   return {"message": "Todos deleted successfully."}
```

Давайте протестируем маршрут `удаления`. Сначала мы добавляем задачу:

```
(venv)$ curl -X POST \
  'http://127.0.0.1:8000/todo' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 1,
  "item": "Example Schema!"
}'
```

Вот ответ:

```
(venv)$ {
  "message": "Todo added successfully."
}
```

Затем удалите задачу:

```
(venv)$ curl -X DELETE \
  'http://127.0.0.1:8000/todo/1' \
  -H 'accept: application/json'
```

Вот ответ:

```
(venv)$ {
  "message": "Todo deleted successfully."
}
```

Давайте проверим, что задача была удалена, отправив запрос `GET` для получениязадачи:

```
(venv)$ curl -X GET \
  'http://127.0.0.1:8000/todo/1' \
  -H 'accept: application/json'
```

Вот ответ:

```
(venv)$ {
  "message": "Todo with supplied ID doesn't exist.
}
```

В этом разделе мы создали приложение с CRUD операциями, объединив уроки,извлеченные из предыдущих разделов. Подтвердив тело запроса, мы смогли убедиться, что в API отправляются правильные данные. Включение параметров пути в наши маршруты также позволило нам получить и удалить одну задачу из нашего списка задач.

1.7. Понимание ответов в FastAPI

Ответы являются неотъемлемой частью жизненного цикла API. Ответы — это отзывы, полученные при взаимодействии с маршрутом API с помощью любого изстандартных методов HTTP. Ответ API обычно предоставляется в формате JSON или XML, но также может быть в форме документа. Ответ состоит из заголовка и тела.

1.7.1. Что такое заголовок ответа?

Заголовок ответа состоит из статуса запроса и дополнительной информации, необходимой для доставки тела ответа. Примером информации, содержащейся взаголовке ответа, является `Content-Type`, который сообщает клиенту возвращаемый тип содержимого.

1.7.2. Что такое тело ответа?

Тело ответа, с другой стороны, представляет собой данные, запрошенные клиентом ссервера. Тело ответа определяется из переменной заголовка `Content-Type` наиболеечасто используемой является `application/json`. В предыдущей главе возвращаемый список задач был телом ответа.

Теперь, когда вы узнали, что такое ответы и из чего они состоят, давайте взглянем накоды состояния HTTP, включенные в ответы в следующем разделе.

1.7.3. Коды состояния

Коды состояния — это уникальные короткие коды, выдаваемые сервером в ответ на запрос клиента. Коды состояния ответа сгруппированы в пять категорий, каждая изкоторых обозначает отдельный ответ:

- `1XX`: Запрос получен.
- `2XX`: Запрос выполнен успешно.
- `3XX`: Запрос перенаправлен.
- `4XX`: Ошибка клиента.
- `5XX`: Ошибка сервера.

Полный список кодов состояния HTTP можно найти по адресу `https://httpstatuses.com/`.

Первая цифра кода состояния определяет его категорию. Общие коды состоянияключают `200` для успешного запроса, `404` для запроса, не найденного и `500` указывающего на внутреннюю ошибку сервера.

Стандартная практика создания веб-приложений, независимо от платформы, заключается в возврате соответствующих кодов состояния для отдельных событий. Код состояния `400` не должен возвращаться в случае ошибки сервера. Точно так жекод состояния `200` не должен возвращаться для неудачной операции запроса.

Теперь, когда вы узнали, что такое коды состояния, давайте научимся строить моделиответов в следующем разделе.

1.8.Построение моделей ответа

В 1.7. установили назначение моделей отклика. Ввы также узнали, как создавать модели с помощью Pydantic. Модели реагирования также построены на Pydantic, но служат другой цели.

В определении путей маршрута мы имеем, например, следующее:

```
@app.get("/todo")
async def retrieve_todo() -> dict:
    return {
        "todos": todo_list
    }
```

Маршрут возвращает список задач, присутствующих в базе данных. Вот пример вывода:

```
{
  "todos": [
    {
      "id": 1,
      "item": "Example schema 1!"
    },
    {
      "id": 2,
      "item": "Example schema 2!"
    },
    {
      "id": 3,
      "item": "Example schema 5!"
    }
  ]
}
```

Маршрут возвращает весь контент, хранящийся в массиве todos. Чтобы указать возвращаемую информацию, нам пришлось бы либо отделить отображаемые данные, либо ввести дополнительную логику. К счастью, мы можем создать модель,содержащую поля, которые мы хотим вернуть, и добавить ее в определение нашегомаршрута, используя аргумент `response_model`.

Давайте обновим маршрут, который извлекает все задачи, чтобы он возвращал массив только элементов задач, а не идентификаторов. Начнем с определения новогокласса модели для возврата списка дел в `model.py`:

```
from typing import List

class TodoItem(BaseModel):
    item: str
    class Config:
        schema_extra = {
            "example": {
                "item": "Read the next chapter of the book"
            }
        }
class TodoItems(BaseModel):
    todos: List[TodoItem]
    class Config:
        schema_extra = {
            "example": {
                "todos": [
                    {
                        "item": "Example schema 1!"
                    },
                    {
                        "item": "Example schema 2!"
                    }
                ]
            }
        }
```

В предыдущем блоке кода мы определили новую модель, `TodoItems`, которая возвращает список переменных, содержащихся в модели `TodoItem` Давайте обновим наш маршрут в `todo.py` добавив в него модель ответа:

```
from model import Todo, TodoItem, TodoItems
…
@todo_router.get("/todo", response_model=TodoItems)
async def retrieve_todo() -> dict:
    return {
        "todos": todo_list
    }
```

Активируйте виртуальную среду и запустите приложение:

```
$ source venv/bin/activate
(venv)$ uvicorn api:app --host=0.0.0.0 --port 8000 --reload
```

Затем добавьте новую задачу:

```
(venv)$ curl -X POST \
  'http://127.0.0.1:8000/todo' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "id": 1,
  "item": "This todo will be retrieved without exposing
   myID!"
}'
```

Получить список дел:

```
(venv)$ curl -X GET \
  'http://127.0.0.1:8000/todo' \
  -H 'accept: application/json'
```

Полученный ответ выглядит следующим образом:

```
{
  "todos": [
    {
      "item": " This todo will be retrieved without
       exposing my ID!"
!"
    }
  ]
}
```

Теперь, когда мы узнали, что такое модели реагирования и как их использовать,мы продолжим использовать их там, где они подходят. Давайте рассмотрим ответы на ошибки и способы обработки ошибок в следующем разделе.

1.9. Обработка ошибок

Ранее мы узнали, что такое коды состояния и как они полезны дляинформирования клиента о состоянии запроса. Запросы могут возвращать ошибочные ответы, и эти ответы могут быть некрасивыми или содержать недостаточную информацию о причине сбоя.

Ошибки запросов могут возникать из-за попыток доступа к несуществующим ресурсам, защищенным страницам без достаточных разрешений и даже из-за ошибок сервера. Ошибки в FastAPI обрабатываются путем создания исключения сиспользованием класса FastAPI's `HTTPException`.

Что такое исключение HTTP?

Исключение HTTP — это событие, которое используется для указания наошибку или проблему в потоке запросов.

Класс `HTTPException` принимает три аргумента:

- `status_code`: Код состояния, который будет возвращен для этого сбоя
- `detail`: Сопроводительное сообщение для отправки клиенту
- `headers`: Необязательный параметр для ответов, требующих заголовков

В наших определениях пути маршрута задачи мы возвращаем сообщение, когда задача не может быть найдена. Мы будем обновлять его, чтобы вызывать `HTTPException`. `HTTPException` позволяет нам вернуть адекватный код ответа на ошибку.

В нашем текущем приложении получение несуществующей задачи возвращает кодстатуса ответа `200` вместо кода статуса ответа `404` на http://127.0.0.1:8000/docs:

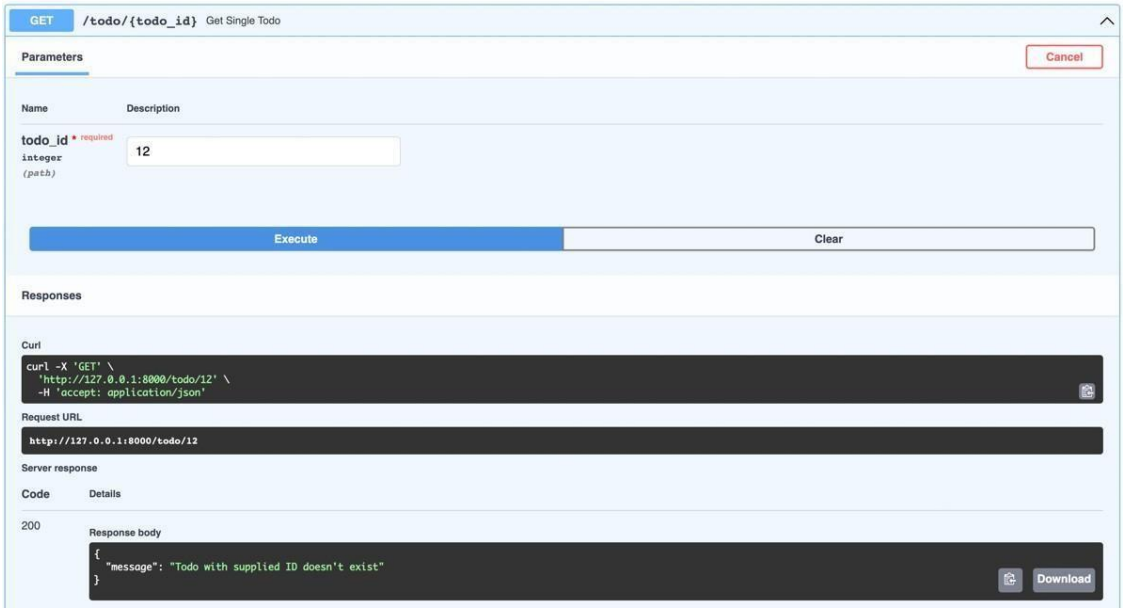


Рисунок 1.6 – Запрос возвращает ответ 200 вместо ответа 404

Обновляя маршруты для использования класса `HTTPException`, мы можем возвращать соответствующие детали в нашем ответе. В `todo.py`,обновите маршруты для получения, обновления и удаления списка дел:

```
from fastapi import APIRouter, Path, HTTPException, status
..
@todo_router.get("/todo/{todo_id}")
async def get_single_todo(todo_id: int = Path(..., title="TheID of
the todo to retrieve.")) -> dict:
for todo in todo_list:
todo.id == todo_id:return {
    "todo": todo
    }
raise HTTPException( status_code=status.HTTP_404_NOT_FOUND,
detail="Todo with supplied ID doesn't exist",
)
```

```
if @todo_router.put("/todo/{todo_id}")
async def update_todo(todo_data: TodoItem, todo_id: int = Path(...,
title="The ID of the todo to be updated.")) -> dict:
for todo in todo_list:
if todo.id == todo_id: todo.item = todo_data.itemreturn {
"message": "Todo updated successfully."
}


raise HTTPException( status_code=status.HTTP_404_NOT_FOUND,
detail="Todo with supplied ID doesn't exist",
)


@todo_router.delete("/todo/{todo_id}")
async def delete_single_todo(todo_id: int) -> dict:for index in
range(len(todo_list)):
todo = todo_list[index]if todo.id == todo_id:
todo_list.pop(index)return {
"message": "Todo deleted successfully."
}
     raise HTTPException( status_code=status.HTTP_404_NOT_FOUND,
detail="Todo with supplied ID doesn't exist",
)
```

Теперь давайте повторим попытку получения несуществующей задачи, чтобыубедиться, что возвращается правильный код ответа:



Рисунок 1.7 – Отображается правильный код ответа 404

Наконец, мы можем объявить код состояния HTTP для переопределения кода состояния по умолчанию для успешных операций, добавив аргумент `status_code` вфункцию декоратора:

```
@todo_router.post("/todo", status_code=201)
async def add_todo(todo: Todo) -> dict:
    todo_list.append(todo)
    return {
        "message": "Todo added successfully."
    }
```

В этом разделе мы узнали, как возвращать правильные коды ответов клиентам, а также переопределять код состояния по умолчанию. Также важно отметить, что код состояния успешный по умолчанию — 200.

Задание

Описанные выше действия опираются на вариант их реализации в среде Linux/Mac. В среде Windows необходимы небольшие изменения в действиях или синтаксисе команд.

При выполнении работы обратите внимание, что код сервера и код клиента необходимо запускать в разных экземплярах командной строки (в разных окнах).

1. Выполнить приведенные действия по созданию простого приложения в FastAPI.
2. Выполнить действия по формированию ответов в соответствии с моделью ответов и действия по обработке ошибок.
3. В пп. 1 и 2 сообщения сделать персонализированными по отношению к разработчику.
4. В соответствии с персональной темой предложить типовые действия в рамках простого приложения FastAPI и модели ошибок. Записать их в форме FastAPI команд.
5. Подготовить отчет.
6. Предоставить или загрузить в СДО: отчет, файлы проекта — `api.py`, `model.py`, `todo.py`,,,.